Loading and Handling Data in R

Data Analytics using R – Chapter 3

Anil

2026-07-30

Challenges of Analytical Data Processing

Data Formats

- Data comes from varied sources (CSV, Excel, SQL, web, etc.)
- R supports many formats via packages (ODBC, readr, etc.)

Data Quality

- Missing values (NA), outliers, inconsistent data
- R provides functions to clean and impute

Project Scope

- Time, cost, and data volume must be planned

Stakeholder Expectation Management

- Present clear results; hide technical details (p-values, etc.)

Expressions in R

Arithmetic Operators

Operation	Operator	Example	Result
Addition	`+`	`4 + 8`	12
Subtraction	`-`	`10 - 3`	7
Multiplication	`*`	`7 * 8`	56
Division	`/`	`8 / 3`	2.666667
Exponentiation	`^` or `**`	`2 ^ 5`	32
Modulus	`%%`	`5 %% 3`	2
Integer division	`%/%`	`5 %/% 2`	2
Square root	`sqrt()`	`sqrt(25)`	5

Logical Values

```
8 < 4
3 * 2 == 5
3 * 2 == 6
F == FALSE
T == TRUE
```

```
[1] FALSE
[1] FALSE
[1] TRUE
[1] TRUE
[1] TRUE
```

Using Logical Operators with Vectors

```
x <- c(1:10)
print(x[1])
print(x>7)
x[(x > 7) | (x < 5)]   # OR
```

```
[1] 1 2 3 4 8 9 10
```

```
x[(x > 7) & (x < 10)]  # AND
```

```
[1] 8 9
```

Dates in R

System Date & Time

```
Sys.Date()
Sys.time()
Sys.timezone()
```

```
[1] "2017-01-13"
[1] "2017-01-13 10:54:37 IST"
[1] "Asia/Calcutta"
```

Formatting and Conversion

```
today <- Sys.Date()
format(today, format = "%B %d %Y")
```

```
[1] "January 13 2017"
```

```
CustomDate <- "2016-01-13"
class(CustomDate)
CustDate <- as.Date(CustomDate)
class(CustDate)
print(CustDate)
```

```
[1] "character"
[1] "Date"
```

Difference Between Dates

```
strDates <- c("08/15/1947", "01/26/1950")
dates <- as.Date(strDates, "%m/%d/%Y")
dates[2] - dates[1]
```

```
Time difference of 895 days
```

Variables

```
Var <- 50
Var
Var + 10
Var / 2
Var <- "R is a Statistical Programming Language"
Var
Var <- TRUE
Var
```

```
[1] 50
[1] 60
[1] 25
[1] "R is a Statistical Programming Language"
[1] TRUE
```

- Variables can be reassigned to different data types.

Functions: `sum()`, `min()`, `max()`

`sum()`

```
sum(1, 2, 3)
sum(1, 5, NA, na.rm = FALSE)
sum(1, 5, NA, na.rm = TRUE)
```

```
[1] 6
[1] NA
[1] 6
```

`min()` and `max()`

```
min(1, 2, 3, NA, na.rm = TRUE)
max(44, 78, 66)
```

```
[1] 1
[1] 78
```

String Functions

`rep()` – Repeat

```
rep("statistics", 3)
```

```
[1] "statistics" "statistics" "statistics"
```

`grep()` – Pattern Search

```
grep("statistical", c("R", "is", "a", "statistical", "language"), fixed = TRUE)
```

```
[1] 4
```

`toupper()` / `tolower()`

```
toupper("statistics")
tolower("STATISTICS")
```

```
[1] "STATISTICS"
[1] "statistics"
```

`substr()` – Extract Substring

```
substr("statistics", 7, 9)
```

```
[1] "tic"
```

Missing Values Treatment

Function	Description
`is.na(x)`	Returns TRUE for missing values
`na.omit(x)`	Removes missing values
`na.exclude(x)`	Removes, but keeps attributes
`na.fail(x)`	Fails if any NA present
`na.pass(x)`	Returns unchanged object

```
is.na(c(4, 5, NA))
na.omit(c(4, 5, NA))
```

```
[1] FALSE FALSE  TRUE
[1] 4 5
attr(,"na.action")
[1] 3
attr(,"class")
[1] "omit"
```

Vectors

Creating Vectors

```
#, eval=FALSE}
c(4, 7, 8)                    # numeric
```

```
[1] 4 7 8
```

```
c("R", "is", "powerful")      # character
```

```
[1] "R"         "is"        "powerful"
```

```
c(TRUE, FALSE, TRUE)          # logical
```

```
[1]  TRUE FALSE  TRUE
```

Sequence Vectors

```
1:5
seq(1, 10, 2)
seq(10, 1, by = -2)
```

```
[1] 1 2 3 4 5
[1] 1 3 5 7 9
[1] 10 8 6 4 2
```

Accessing Elements

```
x <- c("R", "is", "a", "programming", "language")
x[1]
x[c(1, 5)]
x[1:4]
```

```
[1] "R"
[1] "R" "language"
[1] "R" "is" "a" "programming"
```

Vector Names and Math

Assigning Names

```
v <- 1:5
names(v) <- c("r", "is", "a", "programming", "language")
v["programming"]
```

```
programming
          4
```

Vector Arithmetic

```
x <- c(4, 7, 8)
x + 1
x * 2
sin(x)
```

```
[1] 5 8 9
[1] 8 14 16
[1] -0.7568025  0.6569866  0.9893582
```

Vector Recycling

```
c(1, 2, 3) + c(4, 5, 6, 7, 8, 9)
```

```
[1]  5  7  9  8 10 12
```

Matrices

Creating Matrices

```r
matrix(1, 3, 4)              # 3x4 matrix filled with 1
a <- seq(10, 90, by = 10)
matrix(a, 3, 3)              # fill column-wise
```

```
     [,1] [,2] [,3]
[1,]   10   40   70
[2,]   20   50   80
[3,]   30   60   90
```

Using `dim()`

```r
a <- 1:9
dim(a) <- c(3, 3)
a
```

```
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

Matrix Access

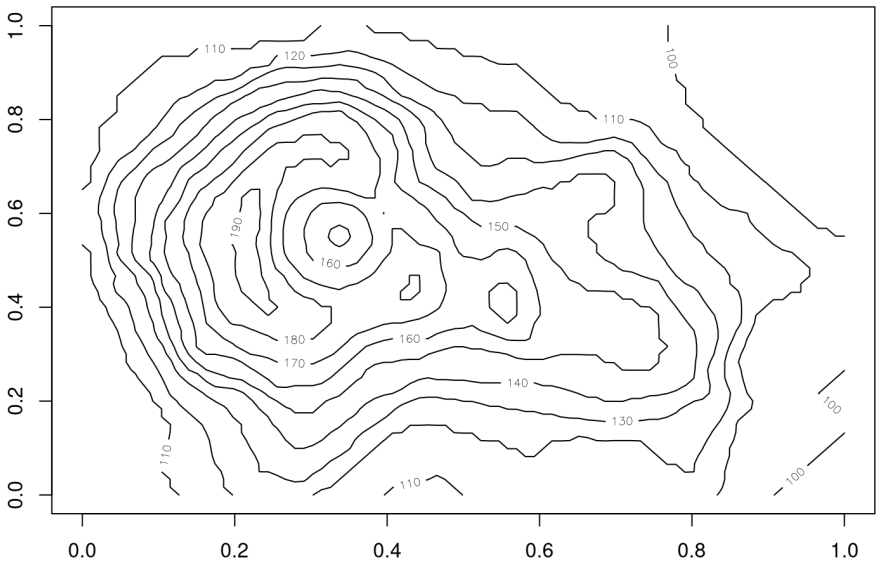
```
mat <- matrix(1:12, 3, 4)
mat
mat[2, 3]       # row 2, col 3
mat[3, ]        # row 3
mat[, 2]        # column 2
mat[, 2:3]      # columns 2 and 3
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
[1] 8
[1] 3 6 9 12
[1] 4 5 6
     [,1] [,2]
[1,]    4    7
[2,]    5    8
[3,]    6    9
```

Visualising Matrices: Contour, Perspective, Heatmap

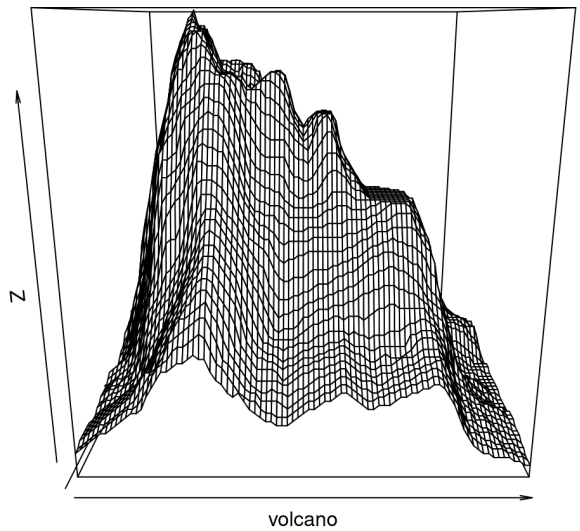
Contour Plot

```
#, eval=FALSE}
contour(volcano)
```



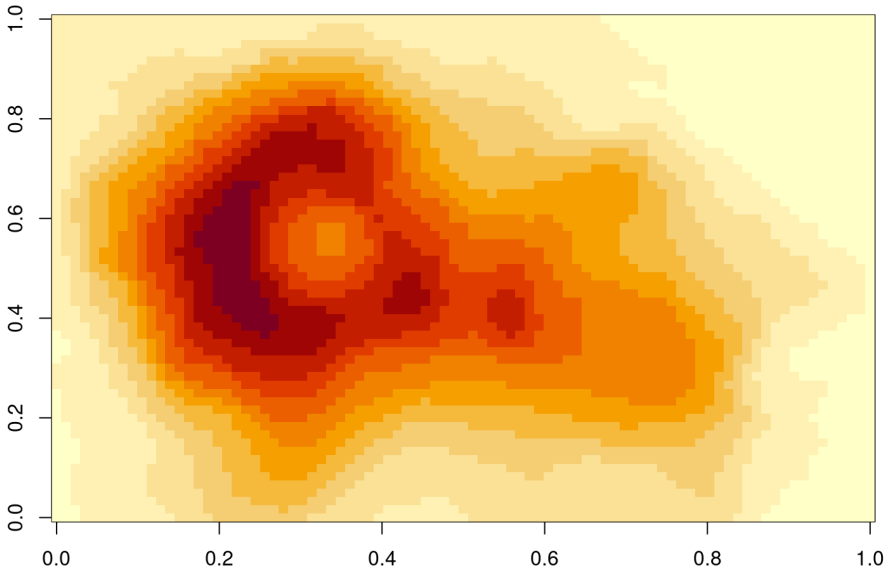
Perspective Plot

```
#, eval=FALSE}
persp(volcano)
```



Heatmap

```
#, eval=FALSE}
image(volcano)
```



Factors

Creating Factors

```
#, eval=FALSE}
HouseColor <- c('red', 'green', 'blue', 'yellow', 'red', 'green', 'blue', 'blue')
types <- factor(HouseColor)
types
```

```
[1] red    green  blue   yellow red    green  blue   blue
Levels: blue green red yellow
```

```
levels(types)
```

```
[1] "blue"   "green"  "red"    "yellow"
```

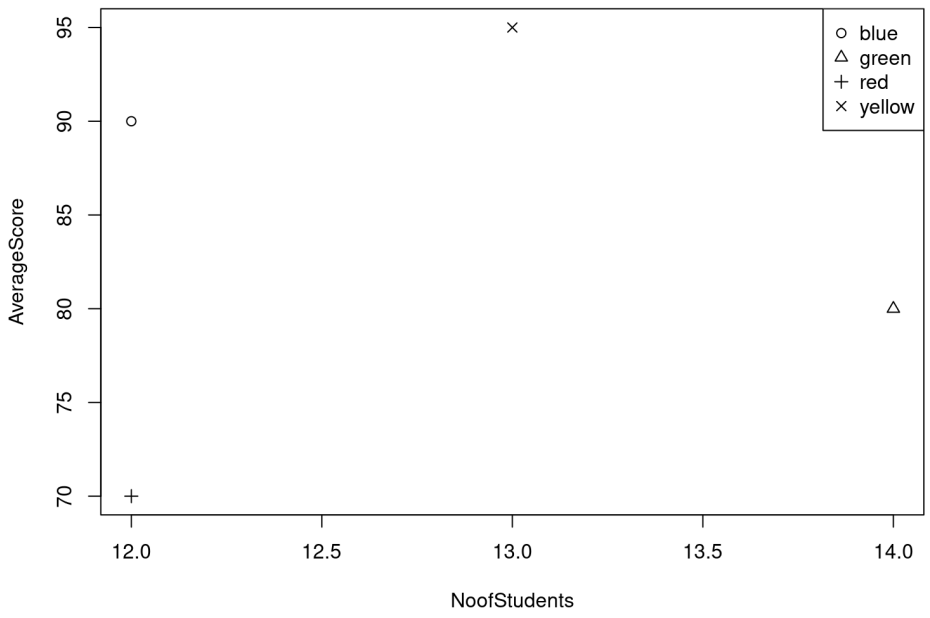
```
as.integer(types)
```

```
[1] 3 2 1 4 3 2 1 1
```

```
[1] red   green blue  yellow red   green blue  blue
Levels: blue green red yellow
[1] "blue"  "green" "red"   "yellow"
[1] 3 2 1 4 3 2 1 1
```

Plotting with Factors

```
#, eval=FALSE}
NoofStudents <- c(12, 14, 12, 13)
AverageScore <- c(70, 80, 90, 95)
plot(NoofStudents, AverageScore, pch = as.integer(types))
legend("topright", levels(types), pch = 1:4)
```



Lists

Creating Lists

```
emp <- list(EmpName = "Alex", EmpUnit = "IT", EmpSal = 55000)
emp
```

```
$EmpName
[1] "Alex"

$EmpUnit
[1] "IT"

$EmpSal
[1] 55000
```

Without Tags

```
EmpList <- list("Alex", "IT", 55000)
EmpList
```

```
[[1]]
[1] "Alex"

[[2]]
[1] "IT"

[[3]]
[1] 55000
```

List Tags and Values

Retrieving Tags and Values

```
names(emp)
unlist(emp)
emp[["EmpName"]]
```

```
[1] "EmpName" "EmpUnit" "EmpSal"
EmpName   EmpUnit    EmpSal
 "Alex"      "IT"    "55000"
[1] "Alex"
```

Add / Delete Elements

```
emp$EmpDesg <- "Software Engineer"
emp$EmpUnit <- NULL   # delete
```

Size of List

```
length(emp)
```

```
[1] 3
```

Recursive List (list within a list)

```
EmpList <- list(emp, emp1)
```

Exploring a Dataset

Function	Description
`names()`	Variable names
`summary()`	Summary statistics
`str()`	Internal structure
`head()`	First n rows (default 6)
`tail()`	Last n rows (default 6)
`class()`	Class of object
`dim()`	Dimensions (rows, columns)
`table()`	Frequency table of categorical variable

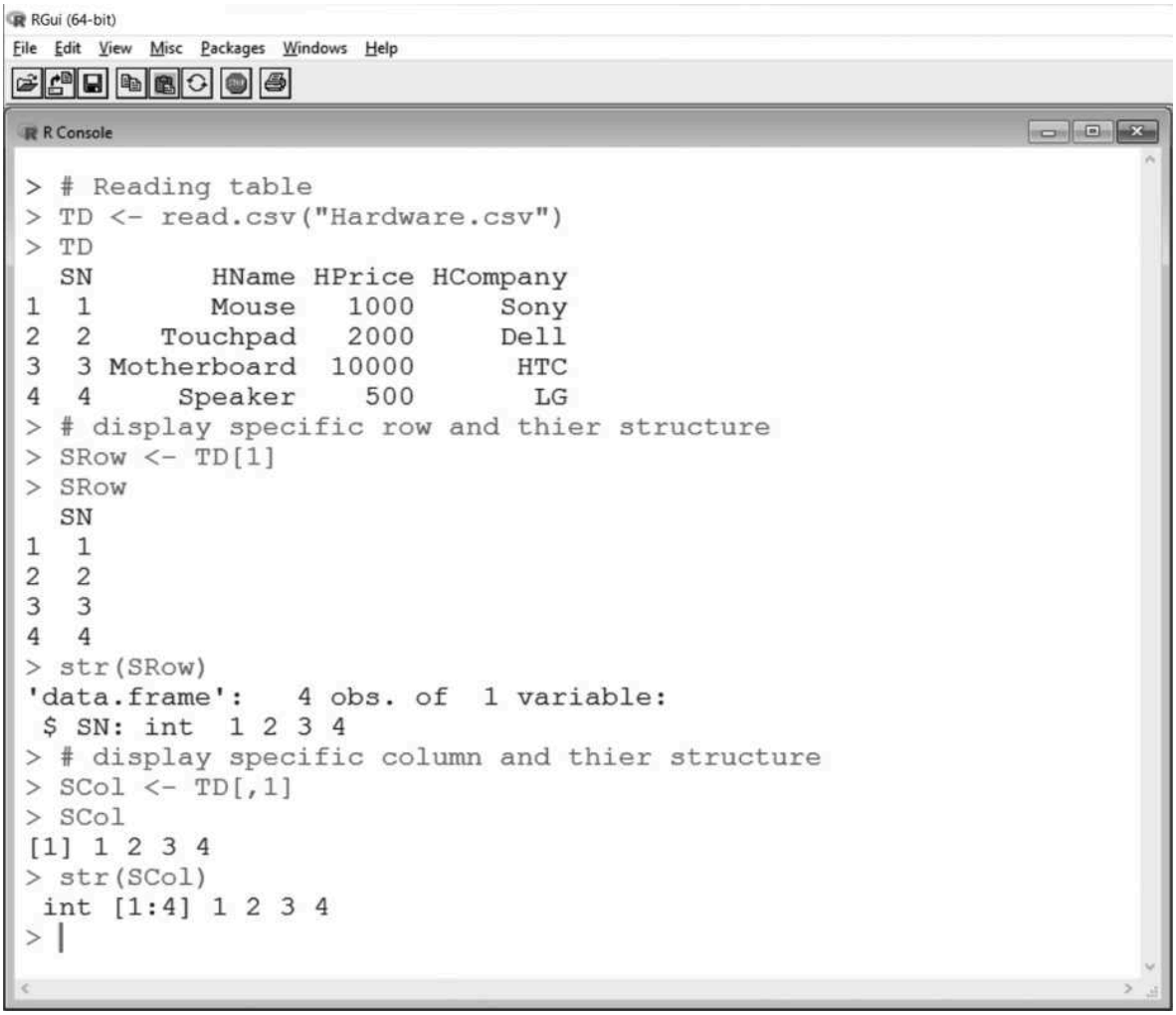
Example on `Orange` dataset

```
data(Orange)
names(Orange)
summary(Orange)
str(Orange)
head(Orange, 3)
tail(Orange, 3)
dim(Orange)
table(Orange$Tree)
```

Conditional Manipulation

```r
# Display first row
TD[1, ]

# Display first column
TD[, 1]
```



Merging Data

`merge()` Function

```
S <- data.frame(RN = c('A','B','E'), RM = c(10,20,30))
T <- data.frame(RN = c('A','B','C'), TM = c(10,20,30))
E <- merge(S, T)   # natural join
E
```

```
  RN RM TM
1  A 10 10
2  B 20 20
```

Join Types

- `all = TRUE` – full outer join
- `all.x = TRUE` – left outer join
- `all.y = TRUE` – right outer join

```
E<-merge(S,T,all.y=TRUE)
E
```

```
  RN RM TM
1  A 10 10
2  B 20 20
3  C NA 30
```

Aggregating and Grouping

aggregate()

```
# Assuming 'Fruit_data.csv' with columns Fruit.Name, Fruit.Price
S <- read.csv("Fruit_data.csv")
aggregate(S$Fruit.Price ~ S$Fruit.Name, S, mean)
```

(See Figure 3.19)

tapply()

```
tapply(S$Fruit.Price, S$Fruit.Name, sum)
```

Simple Analysis Workflow

Input

```
Fruit <- read.csv("Fruit_data.csv")
```

Describe Data Structure

```
names(Fruit)
str(Fruit)
summary(Fruit)
head(Fruit)
tail(Fruit)
```

Describe Variables

```
table(Fruit$Fruit.Name)
hist(Fruit$Fruit.Price)
boxplot(Fruit$Fruit.Price ~ Fruit$Fruit.Name)
plot(Fruit$Fruit.Price, Fruit$Fruit.Quantity)
```

Reading CSV and Spreadsheets

CSV

```
read.csv("Hardware.csv")
read.table("Hardware.csv", header = TRUE, sep = ",")
```

Excel (using `xlsx` package)

```
install.packages("xlsx")
library(xlsx)
read.xlsx("Softdrink.xlsx", 1)   # sheet index
```

Practical Example: SampleSuperstore

```
InputData <- read.csv("d:/SampleSuperstore.csv")
GroupedInputData <- aggregate(InputData$Sales ~ InputData$Category, InputData, sum)
GroupedInputData
```

```
  InputData$Category InputData$Sales
1         Furniture       156514.4
2   Office Supplies        132600.8
3        Technology       168638.0
```

Reading from Packages

`library()` and `data()`

```
library(datasets)
data()                # list all datasets
data(Orange)          # load Orange dataset
Orange
```

Reading Web Data

Packages for Web Data

- `RCurl` – download files, web scraping
- `XML` – parse XML/HTML
- `WDI` – World Bank data
- `quantmod` – Yahoo Finance

Web Scraping Example

```
library(RCurl)
library(XML)
url <- "http://example.com"
html <- getURL(url)
parsed <- htmlTreeParse(html, useInternalNodes = TRUE)
```

Reading JSON

Using `rjson` package

```
install.packages("rjson")
library(rjson)
output <- fromJSON(file = "d:/Jsondoc.json")
output
JSONDataFrame <- as.data.frame(output)
JSONDataFrame
```

Sample JSON data:

```
{
  "EMPID": ["1001", "2001", ...],
  "Name": ["Ricky", "Danny", ...],
  "Dept": ["IT", "Operations", ...]
}
```

Output:

```
  EMPID    Name       Dept
1 1001   Ricky         IT
2 2001   Danny Operations
...
```

Reading XML

Using XML package

```
install.packages("XML")
library(XML)
output <- xmlParse(file = "d:/XMLFile.xml")
rootnode <- xmlRoot(output)
rootsize <- xmlSize(rootnode)   # number of nodes
xmlToDataFrame("d:/XMLFile.xml")
```

Sample XML:

```
<RECORDS>
  <EMPLOYEE>
    <EMPID>1001</EMPID>
    <EMPNAME>Merrilyn</EMPNAME>
    ...
  </EMPLOYEE>
</RECORDS>
```

Output data frame:

```
  EMPID  EMPNAME       SKILLS           DEPT
1 1001 Merrilyn        MongoDB ComputerScience
2 1002   Ramya PeopleManagement HumanResources
...
```

R GUIs for Data Input

GUI	Description
RCommander (Rcmdr)	Simple interface, many plug-ins
Rattle	Data mining GUI
RKward	Transparent front-end, cross-platform
JGR	Java GUI for R
Deducer	Menu system, works with JGR

Using R with Databases

RODBC – MS Access / SQL Server

```
library(RODBC)
conn <- odbcConnect(dsn = "servername", uid = "", pwd = "")
query <- "SELECT * FROM table"
data <- sqlQuery(conn, query)
odbcClose(conn)
```

MySQL – RMySQL

```
library(RMySQL)
conn <- dbConnect(MySQL(), user = "", password = "", dbname = "", host = "")
res <- dbSendQuery(conn, "SELECT * FROM table")
data <- fetch(res, n = -1)
dbDisconnect(conn)
```

PostgreSQL – RPostgreSQL

Similar functions: `dbConnect()`, `dbDisconnect()`.

SQLite – RSQLite

```
library(RSQLite)
conn <- dbConnect(SQLite(), dbname = "myDB.sqlite")
```

JasperDB and Pentaho

JasperReports Server

- Java library: **RevoConnectR**
- Web services: **RevoDeployR**

Pentaho Data Integration (PDI)

- **R Script Executor** – integrates R with PDI
- Allows using R scripts within ETL pipelines

Log Analysis

Reading Log Files

```
LOGS <- read.table("log.log", sep = " ", header = FALSE)
head(LOGS)
```

Analysis and Visualisation

```
summary(LOGS)
barplot(table(LOGS[, column]))
```

Save Plot

```
png("test.png", width = 1024, height = 1024)
barplot(table(LOGS[, column]))
dev.off()
```

Dataset

OrderID,CustomerName,City,Product,Category,Quantity,Price,OrderDate,PaymentMode,OrderStatus
1001,Rahul,Hyderabad,Laptop,Electronics,1,65000,2026-07-01,UPI,Delivered
1002,Sneha,Chennai,Mouse,Electronics,2,800,2026-07-01,Card,Delivered
1003,Ajay,Bangalore,Keyboard,Electronics,1,1500,2026-07-01,Cash,Pending
1004,Kiran,Hyderabad,Monitor,Electronics,1,12000,2026-07-01,UPI,Delivered
1005,Anita,Mumbai,Mobile,Electronics,1,28000,2026-07-01,Card,Delivered
1006,Ramesh,Delhi,Headphones,Electronics,2,2500,2026-07-01,UPI,Cancelled
1007,Pooja,Hyderabad,Printer,Electronics,1,8500,2026-07-01,Card,Delivered
1008,Varun,Chennai,Laptop,Electronics,1,62000,2026-07-01,Card,Delivered
1009,Divya,Bangalore,Mouse,Electronics,3,800,2026-07-01,UPI,Delivered
1010,Arjun,Delhi,Tablet,Electronics,1,22000,2026-07-01,Cash,Pending
1011,Kavya,Mumbai,Keyboard,Electronics,2,1500,2026-07-01,UPI,Delivered
1012,Ravi,Hyderabad,Mobile,Electronics,1,30000,2026-07-01,Card,Delivered
1013,Sita,Chennai,Monitor,Electronics,1,12500,2026-07-01,Cash,Delivered
1014,Meena,Bangalore,Printer,Electronics,1,9000,2026-07-01,UPI,Delivered
1015,Amit,Delhi,Laptop,Electronics,1,67000,2026-07-01,Card,Delivered
1016,Priya,Hyderabad,Headphones,Electronics,2,2500,2026-07-01,UPI,Pending
1017,Nikhil,Mumbai,Mouse,Electronics,1,800,2026-07-01,Cash,Delivered
1018,Deepa,Chennai,Mobile,Electronics,1,29000,2026-07-01,UPI,Delivered
1019,Harish,Bangalore,Tablet,Electronics,1,23000,2026-07-01,Card,Delivered
1020,Latha,Delhi,Monitor,Electronics,1,11800,2026-07-01,Cash,Delivered

Sample amazon data set

Answer the questions

- Which city generated the highest revenue?
- Which product sold the most?
- How many orders are pending?
- Which customers spent more than ₹10,000?
- Are there any missing values in the dataset?
- Find customers from Hyderabad
- Orders with price greater than ₹20,000
- Count orders city-wise
- Total Revenue
- Highest Price Product

insurance dataset

CustomerID,Age,Gender,Employment,Income,MaritalStatus,State,GasUsage,Vehicles,HealthInsurance
1001,25,Male,Yes,28000,Single,Telangana,40,1,Yes 1002,32,Female,Yes,45000,Married,Andhra Pradesh,65,2,Yes 1003,41,Male,Yes,62000,Married,Karnataka,90,2,Yes
1004,29,Female,No,18000,Single,Tamil Nadu,30,0,No
1005,38,Male,Yes,55000,Married,Maharashtra,110,2,Yes
1006,47,Female,Yes,72000,Married,Delhi,150,3,Yes
1007,52,Male,No,25000,Widowed,Telangana,60,1,No
1008,36,Female,Yes,48000,Married,Kerala,80,2,Yes
1009,28,Male,Yes,32000,Single,Karnataka,55,1,No
1010,44,Female,Yes,68000,Married,Telangana,120,2,Yes 1011,33,Male,Yes,51000,Single,Tamil Nadu,75,1,Yes 1012,26,Female,No,20000,Single,Andhra Pradesh,35,0,No
1013,39,Male,Yes,58000,Married,Maharashtra,95,2,Yes
1014,54,Female,Yes,85000,Widowed,Delhi,170,3,Yes
1015,45,Male,Yes,76000,Married,Karnataka,145,2,Yes
1016,31,Female,Yes,43000,Single,Kerala,70,1,Yes
1017,37,Male,Yes,54000,Married,Telangana,88,2,Yes 1018,27,Female,No,21000,Single,Tamil Nadu,45,0,No 1019,49,Male,Yes,70000,Married,Maharashtra,160,3,Yes
1020,35,Female,Yes,47000,Married,Andhra Pradesh,82,2,Yes
1021,42,Male,Yes,-5000,Married,Karnataka,90,2,Yes
1022,120,Female,Yes,56000,Widowed,Delhi,130,2,Yes
1023,0,Male,No,30000,Single,Kerala,40,1,No
1024,34,Female,,50000,Married,Telangana,85,2,Yes
1025,46,Male,Yes,,Married,Maharashtra,150,3,Yes 1026,29,Female,Yes,39000,Single,,60,1,No
1027,40,Male,Yes,64000,Married,Karnataka,,2,Yes
1028,51,Female,No,27000,Widowed,Delhi,95,,No
1029,30,Male,Yes,46000,Single,Telangana,72,1,Yes
1030,58,Female,Yes,980000,Married,Maharashtra,520,4,Yes

Explore the data to find outliers.

What type of customers do we have? Young? Old? Rich? Poor?

Is income normally distributed?

How many customers are Married, Single, Widowed?

Does income depend on age?

Find Customers Without Insurance?